QuantumGuard: Post-Quantum, Tamper-Evident Provenance Logging and Attack Reconstruction Using RiskAdaptive Anchoring

Project report submitted in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Vasudev B AM.SC.U4CSE23267

Vighnesh B AM.SC.U4CSE23269

Under the guidance of

Dr. Devi Rajeev

Center for Cybersecurity Systems and Networks



AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS, KOLLAM 690525

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "QuantumGuard: Post-Quantum, Tamper-Evident Provenance Logging and Attack Reconstruction Using RiskAdaptive Anchoring", submitted by Vasudev B (AM.SC.U4CSE23267) and Vighnesh B (AM.SC.U4CSE23269), in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Computing, Amritapuri, during Semester 7 of the academic year 2026–2027.

Dr. Devi Rajeev Project Guide	Project Coordinator
Chairperson, CSE	Reviewer(s)

Place: Amritapuri Date: _____

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Amrita School of Computing, Amritapuri Campus

DECLARATION

We, Vasudev B (AM.SC.U4CSE23267) and Vighnesh B (AM.SC.U4CSE23269), hereby declare that the project entitled "QuantumGuard: Post-Quantum, Tamper-Evident Provenance Logging and Attack Reconstruction Using RiskAdaptive Anchoring" is a record of our original work carried out under the guidance of Dr. Devi Rajeev, Center for Cybersecurity Systems and Networks, Amrita Vishwa Vidyapeetham, Amritapuri. To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, fellowship or similar qualification to any candidate at any university. All sources and contributions of others have been appropriately acknowledged.

Place: Amritapuri

Date: _____

Student details	**Signature of the student**
Vasudev B AM.SC.U4CSE23267	_____
Vighnesh B AM.SC.U4CSE23269	_____

Project Guide: Dr. Devi Rajeev

Signature of the Project Guide: _____

ACKNOWLEDGEMENT

We thank our guide, Dr. Devi Rajeev of the Center for Cybersecurity Systems and Networks, for her supervision across every stage of this work. Her questions about how the learned component reaches a decision changed the direction of the detection design, and her insistence that each claim be backed by a measurement rather than an argument shaped how the evaluation chapter was written.

We thank the Project Coordinator and the review panel for their feedback at each project review, and the Chairperson of the Department of Computer Science and Engineering for the support given to the project.

We thank Amrita School of Computing, Amritapuri, and Amrita Vishwa Vidyapeetham for giving us the chance to take on this project as part of our curriculum. We are also grateful to the open-source communities behind Linux, eBPF, NetworkX and NumPy, whose work made it possible to build and test this system.

Finally, we thank our families and our classmates for their patience during the long debugging sessions that this project involved.

ABSTRACT

Systems that investigate host intrusions read provenance records, which describe which process started which other process and which files were touched. Every such system reviewed for this work treats its input record as truthful. An attacker who reaches administrative privilege can edit that record before an investigator ever opens it, so a conclusion drawn from a doctored record is confidently wrong, and evidence built on it fails the reliability test that a court applies. This project addresses that exposure by protecting the record itself rather than the analysis performed on it.

The objective was to build a working host provenance system in which event loss is provable, deliberate editing is detectable, the stored evidence is sealed under signatures that remain meaningful once quantum computers exist, and the amount of trustworthy evidence surviving a compromise can be stated as a number. Seven components were implemented. A kernel probe written against the eBPF interface records process execution with monotonically increasing sequence numbers and issues a signed certificate declaring exactly which numbers were observed. Cleaned events become a directed provenance graph. Editing is detected by ten deterministic structural rules working alongside a two-layer GraphSAGE encoder trained through self-supervised corruption. The graph is divided into partitions, each sealed under a Merkle root, chained to its predecessor and signed twice, once with Ed25519 and once with the lattice-based ML-DSA-65 scheme standardised as FIPS 204. A six-signal risk score decides which partitions are anchored to an external ledger and when, so that cost follows suspicion instead of volume.

Across fifteen independent trials per condition, every tampering strategy was detected. The result that justifies the two-layer design is a careful reordering attack, which the rule engine missed in all fifteen trials and the learned encoder caught in all fifteen. Cryptographic verification caught every attack without any training data. Clean captures consumed no external anchoring transactions, while compromised captures anchored one hundred and eighty-nine partitions using sixteen transactions. Against an attacker who both deleted and fabricated relations, sealing raised recovery accuracy from 80.80 per cent to 100.00 per cent by rejecting all seventy-two fabricated relations.

The work contributes to United Nations Sustainable Development Goal 16, Peace, Justice and Strong Institutions, and specifically to target 16.6 on effective, accountable and transparent institutions. Digital evidence that cannot be shown to be unaltered weakens accountability processes that depend on it. The intended beneficiaries are incident response teams, auditors and judicial processes that must rely on machine-generated records. The detection figures reported here are measured on tampering generated by the project's own attack simulator, since no public corpus of poisoned provenance graphs exists; validation against an independent corpus remains future work and is not claimed here.

Keywords: provenance integrity; tamper-evident logging; graph neural network; post-quantum signature; digital forensics

Table of Contents

Acknowledgement i

Abstract ii

List of Figures v

List of Tables vi

1 Introduction 1

1.1 Background and motivation 1

1.2 Objectives 3

1.3 Scope and constraints 4

1.4 Project report organization 5

2 Problem Definition 6

2.1 Problem statement 6

2.2 Existing system and limitations 7

3 Literature Review 8

3.1 Review scope and selection 8

3.2 Descriptive review of literature works 8

3.3 Research gap 14

3.4 Comparison of reviewed literature 15

4 Requirements 18

4.1 Functional requirements 18

4.2 Non-functional requirements 19

4.3 Hardware requirements 20

4.4 Software and data requirements 20

5 Proposed Methodology **22**

5.1 Design (architecture) or conceptual framework 22

5.2 Proposed approach and procedure 23

6 Results and Analysis **29**

6.1 Evaluation setup 29

6.2 Evaluation criteria 29

6.3 Result 30

6.4 Comparative Analysis 34

6.5 Measurement validity and corrections 35

6.6 Discussion and Interpretation 36

6.7 Limitations 36

7 Conclusion **38**

7.1 Conclusion 38

7.2 Future work 39

References **40**

A Source Code and Reproducibility **42**

A.1 Setup and execution 42

A.2 Supporting material 43

List of Figures

5.1 System architecture. Each phase consumes the output of the phase above it. Detection output drives the risk score, which drives anchoring. 22

5.2 Sealing structure for one partition. Leaf relations are hashed under one domain separator and internal nodes under another. The partition root incorporates the previous partition hash and is signed under both schemes. 26

6.1 Detection of attacks held out from training. The learned detector degrades to between 7 and 77 per cent, the rule engine to near zero, and the cryptographic layer holds at 100 per cent on all seven. 31

6.2 Forensic Survivability Index under increasing evidence destruction. The index is evaluated for edge removal, node removal and contiguous window removal at destruction levels of 0, 15, 30, 50 and 75 per cent. 34

List of Tables

1.1 Sustainable Development Goals addressed by the project 2

1.2 Project objectives, evaluation methods and deliverables 3

3.1 Comparison of reviewed literature 15

5.1 Configuration of the learned detector 25

6.1 Detection rate on generated graphs, attacks represented in training, 30 trials per condition 30

6.2 Detection rate on generated graphs, attacks never seen in training, 30 trials per condition 31

6.3 Detection rate on real captures, 3 graphs per condition 32

6.4 Direct attacks on sealed evidence and on the anchor ledger 33

6.5 Risk scoring and anchoring on a real capture of 354 relations 33

6.6 Survivability of a real capture under 15 per cent edge removal 34

6.7 Each layer against the prior art it corresponds to 35

A.1 Repository layout 42

Chapter 1

Introduction

1.1 Background and motivation

When an organisation discovers that one of its machines has been broken into, the first question asked is how the intruder got in and what they reached. That question is answered from provenance data: a record of which process launched which other process, which binary each process executed, and which files were read or written. Modern investigation systems turn this record into a directed graph and search it for the chain of events that explains the compromise. Kairos [1] builds such a graph from whole-system provenance and produces a summary of the attack. MGDA [2] applies a masked graph autoencoder and matches the result against known adversary behaviour patterns. A survey of the field [3] catalogues more than thirty such systems.

These systems share an assumption that is rarely stated. They assume the provenance record handed to them is an honest account of what happened. That assumption holds only while the attacker lacks the privilege needed to edit it. In practice, host compromise and privilege escalation are the same event more often than not, and an attacker holding administrative rights can open the audit log and remove the lines that describe their own activity. Nothing in the investigation pipeline notices, because every component downstream was built to read the record, not to question it.

Three forms of editing matter. Deletion removes the evidence of an action. Reordering changes which event appears to have caused which, and therefore changes the attribution. Fabrication inserts activity that never occurred, which can redirect an investigation towards an innocent account or an unrelated process. All three leave the record syntactically valid. A log file with lines removed is still a well-formed log file.

The consequence reaches beyond the technical. Where an incident leads to a legal or regulatory process, the record becomes evidence, and evidence must be shown to be what it claims to be. A record that cannot be demonstrated to be unaltered since the moment of capture is weak evidence, and an opposing party needs only to raise the possibility of editing to damage it. The question is not whether an investigator believes the log; it is whether the log can be shown to be untouched.

Work on capture quality has advanced independently. eAudit [4] demonstrates that high-rate audit collection with low overhead is achievable using eBPF, and reduces event loss substantially. It does not, however, produce a signed statement that no events were dropped, so a gap in the record remains

ambiguous between a collection failure and a deletion. Transparency log research offers the opposite half of the problem. Merkle^2 [5] shows how an append-only log can support efficient proofs that a specific entry was included, but it is not provenance-aware and uses classical cryptography only. Baseri and colleagues [6] analyse the risk that quantum computation poses to long-lived records and propose anchoring evidence selectively according to assessed risk, but their contribution is a policy analysis without an implementation.

The gap this project occupies lies between these bodies of work. Capture systems do not prove completeness, investigation systems do not question their input, and transparency and post-quantum work has not been applied to host provenance.

1.1.1 Sustainable Development Goal mapping

Table 1.1 sets out the goals and targets this project addresses, how the project maps onto each, who benefits, and what this report demonstrates as opposed to what is only expected at larger scale.

Table 1.1: Sustainable Development Goals addressed by the project

SDG and target	**Mapping to the project**	**Beneficiaries**	**Demonstrated against expected**
SDG 16 Peace, Justice and Strong Institutions; target 16.6 (effective, accountable and transparent institutions)	Accountability for digital conduct rests on machine records. This project attests their completeness and seals their contents, so a third party can check them without trusting the host that produced them.	Incident response teams; internal and external auditors; organisations answerable for system conduct.	Demonstrated: 100 per cent tamper detection by verification alone; signed completeness certificates that fail verification once edited. Expected: the same guarantee at production capture rates.
SDG 16; target 16.3 (rule of law and equal access to justice)	Digital evidence is useful only if its integrity can be shown. Verification produces a pass or fail result from the evidence file and a public key alone, which reduces dependence on the credibility of whoever collected it.	Judicial processes receiving digital evidence; parties contesting evidence they did not collect.	Demonstrated: independent verification with a distinct exit status; inclusion proofs for single relations. Expected, not tested: acceptance in a legal proceeding.
SDG 9 Industry, Innovation and Infrastructure; target 9.1 (reliable and resilient infrastructure)	Survivability is quantified rather than assumed, so an operator can state how much trustworthy evidence remains after a host is compromised.	Platform and security engineers operating Linux hosts.	Demonstrated: five measures across five damage models; sealing raised recovery accuracy by 19.20 points. Expected: similar behaviour on larger fleets.

The mapping is explained in prose below.

This project maps to United Nations Sustainable Development Goal 16, Peace, Justice and Strong Institutions. Two targets apply.

Target 16.6 calls for effective, accountable and transparent institutions at all levels. Institutional accountability for digital conduct rests on machine-generated records of what systems did and who directed them. When those records can be silently edited by whoever holds administrative access, accountability becomes a matter of trusting the administrator. This project produces records whose

completeness is attested and whose contents are sealed under signatures that a third party can check without trusting the machine that produced them. The problem addressed, the objectives stated in Section 1.2 and the implemented verification interface all serve this target directly.

Target 16.3 concerns the rule of law and equal access to justice. Digital evidence is now routine in both criminal and civil matters, and its usefulness depends on whether its integrity can be demonstrated. A verification procedure that any party can run, producing a pass or fail result from the evidence file and a public key alone, reduces the dependence on the credibility of the party who collected it.

The intended beneficiaries are incident response teams who need to defend their findings, internal and external auditors who must rely on system records they did not collect, and judicial processes that receive digital evidence from a party with an interest in the outcome. A secondary alignment exists with Goal 9, target 9.1, on resilient infrastructure, since the survivability measures in Chapter 6 quantify how much usable evidence remains after a host has been compromised.

1.2 Objectives

Five objectives were defined. Table 1.2 links each objective to the method used to evaluate it and to the deliverable it produces, and each is then stated in full.

Table 1.2: Project objectives, evaluation methods and deliverables

No.	Objective	Evaluation method	Deliverable
O1	Capture process execution so that the loss of any event is provable	Sequence continuity across a simulated outage; an edited certificate must fail verification	Capture module; signed completeness certificate
O2	Build a provenance graph whose causal structure matches the real process tree	One identity per process instance; schema contract holds on every edge	Normalisation and graph modules
O3	Detect deliberate editing under four strategies at two levels of attacker care	Detection rate over 15 independent trials per condition	Rule engine; learned detector; detection report
O4	Seal the graph so modification is detectable by cryptography alone, under quantum-resistant signatures	Verification detection rate with the trained model absent	Sealing module; verification command
O5	Anchor evidence externally in proportion to risk and quantify surviving evidence	Ledger transactions consumed, clean against compromised; five measures over five damage models	Anchoring service; local ledger; evaluation report

Objective 1. Capture host process execution such that the loss of any event is provable rather than assumed. Evaluated by restarting capture across a simulated outage and checking that the sequence continuity is preserved and the downtime is recorded explicitly, and by editing a completeness certificate and confirming that verification fails. Deliverable: the capture module and a signed completeness certificate.

Objective 2. Construct a provenance graph whose causal structure reflects the real process tree. Evaluated by checking that each process appears under exactly one identity and that every edge satisfies the graph schema contract. Deliverable: the normalisation and graph modules.

Objective 3. Detect deliberate editing of the provenance record under four attack strategies, deletion, reordering, forgery and timestamp manipulation, at two levels of attacker care. Evaluated by detection rate over fifteen independent trials per condition. Deliverable: the rule engine, the learned detector and the detection report.

Objective 4. Seal the graph so that any later modification is detectable by cryptographic means alone, without reference to any trained model or statistical assumption, and under a signature scheme that remains valid against a quantum adversary. Evaluated by the detection rate of the verification routine across the same attack conditions. Deliverable: the sealing module and the verification command.

Objective 5. Anchor sealed evidence externally in proportion to assessed risk, and quantify how much trustworthy evidence survives a compromise. Evaluated by comparing ledger transaction consumption between clean and compromised captures, and by measuring five survivability indicators across five damage models. Deliverable: the anchoring service, the local ledger and the evaluation report.

1.3 Scope and constraints

The intended users are security engineers and forensic analysts operating on Linux hosts, together with any third party who must verify evidence they did not collect. The system is a protection and verification layer. It is not an intrusion detection system and does not attempt to identify malicious behaviour; it identifies interference with the record of behaviour.

Included functions are kernel-level capture of process execution, completeness attestation, event normalisation, provenance graph construction, detection of record tampering, cryptographic sealing, risk-scored external anchoring, independent verification and survivability measurement.

Four exclusions are deliberate and are stated here rather than left to be discovered. Capture observes process execution through the `sys_enter_execve` tracepoint only; file read and write operations and network connections are not recorded, so the resulting graph is a process tree with file execution targets rather than a data-flow graph. Attack reconstruction, meaning the automatic derivation of an attack narrative from the graph, is not implemented; systems such as Kairos [1] and ATLAS [7] perform that task, and this work protects the record those systems consume. The key store encrypts private keys at rest using scrypt key derivation and AES-256-GCM wrapping, but it is a software component and not a hardware security module, so keys enter host memory during signing. The external ledger implemented and measured here is local, append-only, hash-chained and signed; a distributed ledger adapter is present as an interface but is not connected to a running network and refuses to return fabricated receipts.

Two assumptions are made. The kernel is assumed to be uncompromised at the moment of capture, since an attacker who controls the kernel can suppress the probe itself. The signing key is assumed to be unavailable to the attacker at the time of sealing; the risk-adaptive anchoring component exists precisely to limit the damage when that assumption later fails.

The main constraint during development was the absence of any public dataset of poisoned provenance graphs. This is discussed in Section 4.4 and drove the self-supervised training approach described in Section 5.2.3.

1.4 Project report organization

Chapter 2 states the problem precisely and examines what existing systems do and where they stop. Chapter 3 reviews twelve research works, compares them against common attributes and identifies the research gap. Chapter 4 records the functional, non-functional, hardware and software requirements. Chapter 5 presents the architecture and describes each of the seven components, the detection rules, the learned encoder and its hyperparameters, and the sealing and anchoring procedures. Chapter 6 reports the evaluation setup, the five experiments and their results, the comparison against the rule-only and crypto-only configurations, the interpretation of those results and the limitations that bound them. Chapter 7 concludes and lists future work. The appendix gives the repository layout, the installation steps and the commands that reproduce every number in this report.

Chapter 2

Problem Definition

2.1 Problem statement

Provenance-based investigation treats the provenance record as ground truth. The problem addressed here is that the record is itself an attackable object, and that no stage of the existing pipeline is responsible for noticing when it has been attacked.

Stated precisely: given a host provenance record R produced by a capture mechanism and later presented to an investigator as R', determine whether $R' = R$, and if not, determine which parts of R' can still be relied upon. Existing systems answer neither question, because they are built to analyse R' rather than to authenticate it.

The adversary assumed is one who has obtained administrative privilege on the host and who has an interest in the investigation reaching the wrong conclusion. The adversary can read and write the stored provenance record and can observe the system while doing so. Four capabilities follow. Deletion removes records of chosen actions. Reordering exchanges the apparent order of events, which alters causal attribution without changing the set of events present. Fabrication inserts records of activity that did not occur. Timestamp manipulation shifts events across time windows, moving them outside the period an investigator is examining. A careless adversary performs one of these crudely and leaves structural inconsistencies. A careful adversary adjusts every dependent field so that the result remains internally consistent.

The stakeholders are the incident responder, whose conclusions depend on the record; the organisation, whose accountability depends on those conclusions; and any external party, auditor, regulator or court, that must assess evidence collected by a party with an interest in the outcome.

The consequences of leaving the problem unaddressed are concrete. An investigation can attribute an intrusion to the wrong account or the wrong process, allowing the real entry point to persist. An organisation can report an incident inaccurately to a regulator. Evidence submitted in a legal proceeding can be excluded or discounted because its integrity cannot be demonstrated, which does not merely weaken one case but weakens the general reliance that can be placed on machine-generated records.

The system takes, as input, process execution events observed at the kernel boundary. It produces, as output, a provenance graph, a tamper determination with the specific violations found, a set of sealed

partitions with signatures and inclusion proofs, a set of external anchor receipts for the partitions whose risk warranted them, and a survivability report stating how much of the evidence would remain trustworthy under a given level of destruction.

2.2 Existing system and limitations

Four families of existing work bear on this problem. Each solves a part and leaves the rest.

Provenance-based intrusion detection. Kairos [1], MGDA [2], threaTrace [8] and UNICORN [9] represent the state of the art in finding attacks within provenance graphs. Their reported accuracy is high. The limitation relevant here is structural rather than a matter of performance: each takes the graph as given. Kairos reconstructs an attack summary from whole-system provenance and reports detection latency, but a deletion performed before Kairos runs simply removes the subgraph it would have found. UNICORN builds a streaming summary of the graph and compares it against learned normal behaviour, which means an attacker who edits the record towards normality is rewarded rather than detected. The survey by Zipperle and colleagues [3] confirms the pattern across the field: record integrity is outside the stated scope of the systems surveyed.

Audit capture. eAudit [4] reduces event loss through an eBPF collection path with tunable prioritisation and reports an overhead of approximately 4.5 per cent across seven stress benchmarks. The limitation is that reducing loss is not the same as proving its absence. When the record contains a gap, an investigator cannot distinguish a dropped event from a deleted one, and the attacker benefits from the ambiguity.

Transparency logs. Merkle^2 [5] provides an append-only log with low-latency append and lookup proofs, and Certificate Transparency [10] established the operational model for publicly verifiable append-only logs. Both are built for entries that arrive as an independent stream. Provenance is a graph whose entries have causal relationships, and partitioning a graph for sealing raises questions, such as which relations belong together and how a whole partition disappearing is detected, that a linear log does not face. Both also use classical hash and signature constructions only.

Post-quantum and risk-adaptive protection. The standardisation of ML-DSA as FIPS 204 [11] makes lattice-based signing deployable. Baseri and colleagues [6] carry out a structured risk assessment of quantum threats and propose anchoring records selectively according to assessed risk together with selective post-quantum verification. Their work is explicitly a policy and risk analysis; no implementation is offered and no measurement of the cost saved by adaptivity is reported.

The gap, therefore, is not a missing algorithm. It is that no system joins these parts into a path that runs from attested capture, through detection of editing, into cryptographically sealed and risk-anchored evidence, with the survivability of that evidence measured at the end.

Chapter 3

Literature Review

3.1 Review scope and selection

Twelve works were reviewed. The search covered the IEEE Xplore, ACM Digital Library and USENIX proceedings collections, together with the NIST publication series for the cryptographic standards used.

Search terms were combined from three groups: provenance terms (*provenance graph*, *audit log*, *system call capture*), integrity terms (*tamper-evident*, *transparency log*, *Merkle*, *post-quantum signature*) and learning terms (*graph neural network*, *anomaly detection*, *inductive representation*).

Works were included when they satisfied three conditions: the work appeared in a peer-reviewed venue or is a published standard; the work addresses host provenance, log integrity or a technique directly adopted in this project; and the work reports its evaluation method, so that its limitations can be stated from the authors' own account rather than inferred. The publication period examined was 2013 to 2025, with the single earlier anchor being the Certificate Transparency specification, which is included because it established the operational model that later transparency log work builds upon. Two foundational works, GraphSAGE [12] and FIPS 204 [11], are included because the project implements them directly and their reported properties constrain the design.

All entries are primary sources. Where a numerical value is quoted below, it is the value reported by the authors; values not reported by the authors are marked as such rather than estimated.

3.2 Descriptive review of literature works

3.2.1 eAudit

Sekar, Kimm and Aich (2024) in "eAudit: A Fast, Scalable and Deployable Audit Data Collection System" [4] address the cost and reliability of collecting audit data on production Linux hosts, where conventional audit subsystems impose overhead high enough that operators disable them. The work proposes a collection system built on eBPF with tunable event prioritisation, so that high-value events survive when the collection path saturates. The study is situated on Linux hosts under seven stress-test workloads including Postmark and Linux kernel compilation. The authors assess the system through runtime overhead measurement, event-loss rate and the width of the window in which log tampering

is possible before records are flushed. They report an overhead of approximately 4.5 per cent, with event loss substantially reduced relative to the baseline audit path. Comparisons are made against the Linux audit subsystem and prior eBPF collectors. The main limitation is that the system offers no signed statement that zero events were dropped, so a gap in the record remains ambiguous. This work informs the present project through its demonstration that eBPF capture is practical at production rates, and through the unresolved gap it leaves: loss reduction without a proof of completeness.

3.2.2 Kairos

Cheng, Lv, Liang, Wang, Sun, Pasquier and Han (2024) in "Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance" [1] address the difficulty of detecting intrusions and reconstructing their narrative from whole-system provenance at practical cost. The work proposes a graph neural network encoder and decoder that learns temporal changes in the provenance graph and emits a compact attack summary graph rather than an alert stream. The study is situated on the DARPA Transparent Computing engagement 3 and engagement 5 datasets and on the OpTC dataset. The authors assess the system through precision, recall and detection latency, together with qualitative inspection of the reconstructed summary graphs. They report detection performance competitive with prior provenance-based detectors while producing summaries small enough for an analyst to read. Comparisons are included against earlier provenance detectors evaluated on the same engagements. The main limitation, stated by the authors as a scope boundary, is that the input provenance graph is assumed complete and untampered. This work informs the present project by defining the class of consumer that the proposed system protects, and by making explicit the assumption that this project treats as the problem.

3.2.3 MGDA

Cui, Jiang, Li, Ma, Zhang, Yang and Feng (2025) in "MGDA: A Provenance Graph-Based Framework for Threat Detection and Attack Scenario Reconstruction" [2] address the combined problem of detecting threats in provenance graphs and reconstructing the attack scenario in terms an analyst can act upon. The work proposes a multi-view masked graph autoencoder whose learned representation is matched against MITRE ATT&CK tactic and technique descriptions, so that detections carry semantic labels. The study is situated on the StreamSpot dataset, the Unicorn Wget dataset and DARPA Transparent Computing engagement 3. The authors assess the framework through precision, recall and F1 score, with scenario reconstruction assessed qualitatively against known ground truth. They report a precision of 97.58 per cent and an F1 score of 98.03 per cent. Comparisons are included against prior graph-learning detectors on the same datasets. The main limitation is that graph tampering falls outside the stated scope and no integrity layer is present. This work informs the present project by showing how strong reported detection performance can be while resting entirely on the honesty of the input.

3.2.4 Provenance-based intrusion detection survey

Zipperle, Gottwalt, Chang and Dillon (2022) in "Provenance-based Intrusion Detection Systems: A Survey" [3] address the absence of a consolidated account of how provenance has been applied to intrusion detection. The work proposes a taxonomy organised by data collection method, graph rep-

resentation, detection technique and evaluation practice, and applies it across the systems surveyed. The study is situated as a secondary study over peer-reviewed systems published up to 2022, with the DARPA Transparent Computing datasets identified as the dominant evaluation corpus. The authors assess the field qualitatively against the taxonomy, reporting which capabilities are widely implemented and which are rare. They report that the surveyed systems concentrate on detection accuracy and scalability, and that provenance data integrity is addressed by very few. Comparisons across systems are the substance of the work. The main limitation is that, being a survey, it offers no implementation and its currency is bounded by its publication date. This work informs the present project by providing independent confirmation, from a systematic review rather than from our own reading, that record integrity is an open gap in the field.

3.2.5 threaTrace

Wang, Wang, Zhou, Sun, Yin, Han, Zhang, Shi and Yang (2022) in "threaTrace: Detecting and Tracing Host-Based Threats in Node Level Through Provenance Graph Learning" [8] address the fact that graph-level detection identifies that a host is compromised without indicating where, which leaves the analyst to search manually. The work proposes a node-level detector built on an inductive graph neural network trained without attack labels, so that individual anomalous nodes are flagged and can be traced. The study is situated on the StreamSpot, Unicorn Wget and DARPA Transparent Computing datasets. The authors assess the detector through node-level precision, recall and F1 score, and through the proportion of attack nodes successfully traced. They report high node-level recall with false positive rates low enough for practical triage. Comparisons are included against graph-level detectors evaluated on the same corpora. The main limitation is that detection quality depends on the training corpus reflecting the deployment environment, and record integrity is not considered. This work informs the present project through its inductive node-level formulation, which is the formulation adopted for the learned component described in Section 5.2.3.

3.2.6 UNICORN

Han, Pasquier, Bates, Mickens and Seltzer (2020) in "UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats" [9] address the detection of slow-moving intrusions whose individual actions appear unremarkable and only become suspicious in aggregate over long periods. The work proposes a streaming detector that maintains an evolving sketch of the whole provenance graph and compares it against a model of normal system behaviour learned without attack data. The study is situated on the StreamSpot dataset, on a purpose-built supply-chain attack dataset and on DARPA Transparent Computing data. The authors assess the detector through precision, recall, accuracy and runtime memory consumption. They report accurate detection of long-running campaigns at bounded memory cost. Comparisons are included against window-based and signature-based alternatives. The main limitation is that the normality model can be shifted by an adversary with sufficient patience, and the record from which the sketch is built is assumed genuine. This work informs the present project by illustrating the specific danger of a detector that rewards normal-looking input: an attacker who edits the record towards normality defeats it without ever touching the detector.

3.2.7 ATLAS

Alsaheel and colleagues (2021) in "ATLAS: A Sequence-based Learning Approach for Attack Investigation" [7] address the manual effort required to reconstruct an attack story from audit logs after an alert has fired. The work proposes a sequence-based learning method that converts causal paths in the audit graph into text-like sequences and learns to separate attack sequences from benign ones, producing a recovered attack narrative. The study is situated on ten realistic attack scenarios executed on instrumented hosts, with audit data collected through standard operating system logging facilities. The authors assess the method through precision and recall of the recovered attack steps and through the reduction in analyst effort. They report accurate recovery of attack sequences from a small number of labelled examples. Comparisons are included against manual investigation and prior causality-analysis tools. The main limitation is dependence on the completeness and correctness of the underlying audit log, together with the labelled attack sequences required for training. This work informs the present project by demonstrating that investigation quality is bounded by log quality, which is the quantity this project protects.

3.2.8 Merkle^2

Hu, Hooshmand, Kalidhindi, Yang and Popa (2021) in "Merkle^2: A Low-Latency Transparency Log System" [5] address the latency cost of transparency logs, in which appending an entry and proving its inclusion have historically traded against one another. The work proposes a nested construction, a Merkle tree built over Merkle trees, that supports both efficient append and efficient lookup proofs without the periodic rebuild that simpler designs require. The study is situated on synthetic transparency-log update workloads at varying scale. The authors assess the system through append latency, lookup latency and proof size. They report latency improvements over prior transparency log constructions at comparable proof sizes. Comparisons are included against earlier append-only log designs. The main limitations are that the construction uses classical hash and signature primitives only, and that it is designed for independent log entries rather than for data with internal causal structure. This work informs the present project through its nested tree idea, which is adapted in Section 5.2.6 to batch many urgent partition roots into a single external anchor.

3.2.9 Certificate Transparency

Laurie (2014) in "Certificate Transparency" [10] address the problem that a certificate authority could issue a fraudulent certificate without the affected domain owner ever learning of it. The work proposes publicly auditable append-only logs of issued certificates, together with signed timestamps and inclusion and consistency proofs that allow any party to verify that a log has not been rewritten. The study is situated in the public web public key infrastructure, with the specification defining log, monitor and auditor roles. The authors assess the design analytically, through the detection guarantees the proofs provide rather than through measured performance. They report that misissuance becomes detectable by any monitor once logs are operated and checked. Comparisons are made against the prior model, in which no external record of issuance existed. The main limitations are that the design detects misissuance after the fact rather than preventing it, and that its guarantees depend on logs being independently monitored. This work informs the present project through its operational model: an external record that an interested party cannot quietly rewrite, which is the role played by

the anchoring ledger in Chapter 5.

3.2.10 Blockchain security risk assessment in the quantum era

Baseri, Hafid, Shahsavari, Makrakis and Khodaiemehr (2025) in "Blockchain Security Risk Assessment in Quantum Era, Migration Strategies, and Proactive Defense" [6] address the exposure of long-lived records, whose cryptographic protection must survive for years, to future quantum attack. The work proposes a structured risk assessment using the STRIDE threat model, from which it derives migration strategies including risk-adaptive anchoring and selective post-quantum verification, so that protection effort is concentrated where assessed risk is highest. The study is situated as an analysis of existing blockchain platforms including Bitcoin, Ethereum, Ripple, Litecoin and Zcash. The authors assess their proposals through qualitative risk assessment against the threat model. They report a prioritised set of migration actions and identify which asset classes face the greatest exposure. Comparisons are drawn between platforms by exposure profile. The main limitation, stated by the authors, is that the work is a policy-level analysis with no working implementation and no measurement of the cost saved by adaptivity. This work informs the present project most directly of all those reviewed: the risk-adaptive anchoring they propose is implemented and measured in Chapter 5 and Chapter 6.

3.2.11 FIPS 204 module-lattice digital signature standard

Dang and colleagues (2024) in "Module-Lattice-Based Digital Signature Standard" [11], published by the National Institute of Standards and Technology, address the need for a digital signature scheme whose security does not rest on problems that a sufficiently large quantum computer could solve. The standard specifies ML-DSA, a signature scheme founded on the hardness of problems over module lattices, in three parameter sets. The study is situated in the NIST post-quantum standardisation process, following multiple rounds of public cryptanalysis. The standard is assessed through that public cryptanalytic review together with specified security category targets for each parameter set. It reports key, signature and performance characteristics for each parameter set, with signatures substantially larger than elliptic-curve equivalents. Comparisons against classical schemes are implicit in those size and speed figures. The main limitation is the relative youth of lattice-based cryptanalysis compared with the decades of scrutiny applied to elliptic curves, which is the stated reason that hybrid deployment is recommended during transition. This work informs the present project through the ML-DSA-65 parameter set, used in the dual-signature construction of Section 5.2.5, and through the recommendation that leads to that construction being hybrid rather than lattice-only.

3.2.12 GraphSAGE inductive representation learning

Hamilton, Ying and Leskovec (2017) in "Inductive Representation Learning on Large Graphs" [12] address the inability of earlier graph embedding methods to produce representations for nodes that were absent during training, which prevents their use on graphs that grow or change. The work proposes GraphSAGE, which learns aggregation functions over a node's sampled neighbourhood rather than learning a fixed embedding per node, making the learned model applicable to unseen nodes and unseen graphs. The study is situated on citation, social and protein interaction benchmarks, with mean, pooling and LSTM aggregators compared. The authors assess the method through classifi-

cation performance on unseen nodes and unseen graphs, together with training and inference cost. They report substantial improvement over transductive baselines in the inductive setting. Comparisons are included against matrix-factorisation embeddings and transductive graph convolution. The main limitation is that neighbourhood sampling introduces variance, and that performance depends on the informativeness of the input node features. This work informs the present project directly: every provenance capture produces a graph that was never seen during training, so an inductive formulation is a requirement rather than a preference, and the mean aggregator variant is the one implemented in Section 5.2.3.

3.3 Research gap

Table 3.1 compares the reviewed works. Reading down the limitations column, three consistent absences appear.

The first is that provenance consumers do not authenticate their input. Kairos, MGDA, threaTrace, UNICORN and ATLAS all improve what can be extracted from a provenance graph, and all five assume the graph is a faithful record. The survey confirms that this holds across the field rather than being a property of the five systems chosen here.

The second is that capture systems reduce event loss without proving its absence. eAudit demonstrates that collection can be fast and reliable, yet a gap in its output cannot be distinguished from a deletion. Completeness is therefore asserted by the operator rather than demonstrated to a verifier.

The third is that integrity techniques exist but have not been applied to this data. Merkle^2 and Certificate Transparency provide the machinery for append-only verifiable records, FIPS 204 provides signatures that survive quantum attack, and Baseri and colleagues propose spending anchoring effort in proportion to risk. None of this has been brought to host provenance, and the risk-adaptive proposal in particular has never been implemented or measured.

The gap addressed by this project is the absence of a path that runs from attested capture, through detection of deliberate editing, into cryptographically sealed and risk-anchored evidence, with the survivability of that evidence measured. The individual components are known. The composition is not, and the composition is where the protection comes from, since each stage covers a failure of the stage before it.

3.4 Comparison of reviewed literature

The twelve reviewed works are compared below against common attributes. Study context covers the setting and evidence base used. Entries marked "Not reported" were not stated by the authors.

Table 3.1: Comparison of reviewed literature

Work and citation	Problem or objective	Approach and contribution	Study context	Evaluation criteria	Key findings	Limitations and gap
eAudit [4] Sekar et al., 2024	Collect audit data at production rates without unacceptable overhead	eBPF collection path with tunable event prioritisation	Linux hosts; seven stress benchmarks including Postmark and kernel build	Runtime overhead; event-loss rate; tampering window	Overhead about 4.5 per cent; event loss substantially reduced	No signed proof that zero events were dropped; a gap stays ambiguous
Kairos [1] Cheng et al., 2024	Detect intrusions and reconstruct the attack narrative from whole-system provenance	Graph encoder and decoder over temporal graph change; emits attack summary graphs	DARPA TC engagements 3 and 5; OpTC	Precision; recall; detection latency	Competitive detection with analyst-readable summaries	Input graph assumed complete and untampered
MGDA [2] Cui et al., 2025	Detect threats and label them with adversary techniques	Multi-view masked graph autoencoder with MITRE ATT&CK matching	StreamSpot; Unicorn Wget; DARPA TC engagement 3	Precision; recall; F1	Precision 97.58 per cent; F1 98.03 per cent	Graph tampering outside scope; no integrity layer
Survey [3] Zipperle et al., 2022	Consolidate how provenance is used for intrusion detection	Taxonomy over collection, representation, detection and evaluation	Secondary study of peer-reviewed systems to 2022	Qualitative assessment against the taxonomy	Field concentrates on accuracy and scale; integrity rarely addressed	Survey only; currency bounded by publication date
threaTrace [8] Wang et al., 2022	Locate which nodes are anomalous, not only which hosts	Inductive graph learning for node-level detection without attack labels	StreamSpot; Unicorn Wget; DARPA TC	Node-level precision, recall, F1; tracing coverage	High node-level recall at practical false positive rates	Depends on training corpus matching deployment; integrity not considered

Table 3.1 (continued)

Work and citation	**Problem or objective**	**Approach and contribution**	**Study context**	**Evaluation criteria**	**Key findings**	**Limitations and gap**
UNICORN [9] Han et al., 2020	Detect slow campaigns whose steps look individually normal	Streaming whole-graph sketch compared against learned normality	StreamSpot; purpose-built supply-chain dataset; DARPA TC	Precision; recall; accuracy; memory use	Detects long campaigns at bounded memory	Normality model shiftable by a patient adversary; record assumed genuine
ATLAS [7] Alsaheel et al., 2021	Reduce manual effort in reconstructing attack stories	Causal paths converted to sequences; sequence model separates attack from benign	Ten realistic attack scenarios on instrumented hosts	Precision and recall of recovered attack steps	Accurate recovery from few labelled examples	Bounded by audit log completeness; needs labelled attack sequences
Merkle^2 [5] Hu et al., 2021	Remove the append and lookup latency trade-off in transparency logs	Nested Merkle-of-Merkles construction with efficient proofs both ways	Synthetic transparency-log update workloads	Append latency; lookup latency; proof size	Latency improved over prior logs at comparable proof size	Classical primitives only; built for independent entries, not causal graphs
Certificate Transparency [10] Laurie, 2014	Make fraudulent certificate issuance detectable	Publicly auditable append-only logs with inclusion and consistency proofs	Public web PKI; log, monitor and auditor roles	Analytical detection guarantees	Misissuance becomes detectable by any monitor	Detects after the fact; depends on logs being monitored
Baseri et al. [6] 2025	Assess quantum exposure of long-lived records and plan migration	STRIDE risk assessment; proposes risk-adaptive anchoring and selective PQ verification	Analysis of Bitcoin, Ethereum, Ripple, Litecoin, Zcash	Qualitative risk assessment	Prioritised migration actions and exposure profiles	Policy analysis only; no implementation; adaptivity never measured
FIPS 204 [11] Dang et al., 2024	Standardise a signature scheme secure against quantum attack	ML-DSA module-lattice signatures in three parameter sets	NIST post-quantum standardisation process	Public cryptanalysis; security category targets	Parameter sets published; signatures larger than elliptic curve	Lattice cryptanalysis less mature; hybrid deployment recommended
GraphSAGE [12] Hamilton et al., 2017	Represent nodes unseen during training	Learned neighbourhood aggregation rather than per-node embeddings	Citation, social and protein interaction benchmarks	Classification on unseen nodes and graphs; cost	Large gains over transductive baselines in the inductive setting	Sampling variance; depends on input feature quality

Summary of the gap. Three patterns run through the twelve works. The five detection and investigation systems, Kairos, MGDA, threaTrace, UNICORN and ATLAS, differ in method but agree in what they take for granted: each reads the provenance record as a faithful account, so none of them asks whether the graph was edited before it arrived. The survey confirms this is a property of the field rather than of the five chosen here. Second, capture work reduces event loss without proving its absence, so eAudit leaves a gap in the record indistinguishable from a deletion. Third, the machinery that would close these gaps already exists but has not been brought to host provenance: Merkle^2 and Certificate Transparency give append-only verifiable records, FIPS 204 gives signatures that survive quantum attack, GraphSAGE gives an inductive encoder for graphs never seen in training, and Baseri and colleagues propose anchoring in proportion to risk without implementing or measuring it. This project addresses all three by composing them, which is where the protection comes from, since each stage covers a failure of the stage before it.

Chapter 4

Requirements

4.1 Functional requirements

FR1. Kernel-level capture. The system shall record every process execution observed at the `sys_enter_execve` tracepoint, with the process identifier, parent identifier, executed image path, user identifier and kernel timestamp. Actor: the capture daemon. Acceptance: a process launched during a capture session appears in the stored event set.

FR2. Sequence assignment. The system shall assign each accepted event a strictly increasing sequence number within a run. Acceptance: for any stored run, the set of sequence numbers contains no duplicate and no unexplained gap.

FR3. Session continuity. The system shall resume sequence numbering across restarts and record the interruption as an explicit session gap attestation. Acceptance: after a stop and restart, the first new sequence number exceeds the last stored one and a gap record naming the downtime exists.

FR4. Completeness certificate. The system shall issue a certificate stating the sequence range observed, the events counted, the gaps attested and whether the range is complete, signed so that a third party can verify it. Acceptance: verification of an unmodified certificate passes and verification of a certificate whose body has been edited fails.

FR5. Normalisation. The system shall remove duplicate events, reject events violating temporal constraints, and infer parent relationships from the execution chain, recording a reason for every discarded event. Acceptance: the discard log accounts for the difference between events read and events retained.

FR6. Graph construction. The system shall build a directed graph of process and file nodes connected by spawn and execute relations, with exactly one node identity per process instance. Acceptance: no process instance appears under two identities and every edge satisfies the schema contract.

FR7. Rule-based tamper detection. The system shall evaluate ten deterministic structural rules against the graph and report each violation with its rule name, severity and the specific node, edge or sequence position concerned. Acceptance: each rule fires on a graph constructed to violate it and does not fire on a clean graph.

FR8. Learned tamper detection. The system shall score nodes and the graph as a whole using a

trained inductive graph encoder and report the nodes exceeding the decision threshold. Acceptance: the model produces scores for a graph it has never seen.

FR9. Sealing. The system shall divide the graph into partitions, compute a Merkle root for each, chain each partition to its predecessor by hash, and sign each root with both an Ed25519 and an ML-DSA-65 signature. Acceptance: verification of an unmodified sealed set passes; verification after any single relation is altered, removed or inserted fails.

FR10. Inclusion proof. The system shall produce, for any sealed relation, a proof of its membership in its partition that can be checked against the partition root alone. Acceptance: a valid proof verifies and a proof with any element altered does not.

FR11. Risk scoring. The system shall compute a risk score per partition from six signals and assign a tier, applying hard escalation when a tamper alert or a blocking rule violation is present. Acceptance: a clean partition scores below the tier 2 threshold; a partition containing a blocking violation is assigned tier 1 regardless of its weighted score.

FR12. External anchoring. The system shall anchor partitions whose tier requires it to an append-only ledger, batching simultaneous urgent partitions under a second-order Merkle tree, and shall record a receipt with an inclusion proof for each anchored partition. Acceptance: every receipt verifies independently against the stored ledger block.

FR13. Verification. The system shall provide a command that verifies signatures, hash chain and graph contents and returns a distinct exit status for pass and fail. Acceptance: exit status 0 on intact evidence and 2 on tampered evidence.

FR14. Survivability measurement. The system shall simulate evidence destruction under five damage models and report the four component measures together with the combined index. Acceptance: every measure decreases as the damage ratio increases.

4.2 Non-functional requirements

NFR1. Verifiability without trust. Verification shall require only the evidence files and the public key. No component of verification shall depend on the host that produced the evidence. Criterion: verification succeeds from a fresh checkout with no database and no captured graph present.

NFR2. Cryptographic independence. Tamper detection by the sealing layer shall not depend on any trained model, threshold or statistical assumption. Criterion: the integrity experiment in Section 6.3.2 runs with the learned model absent.

NFR3. Capture overhead. The capture path shall not require user-space processing in the hot path; events shall be moved through a kernel ring buffer and processed asynchronously. Criterion: the probe writes only to the ring buffer and performs no formatting in kernel context.

NFR4. Reproducibility. Every numerical result in this report shall be regenerable by a single documented command with a fixed random seed. Criterion: repeated runs at the same seed produce identical detection counts.

NFR5. Graceful degradation. The system shall operate without a database, without root privilege and without a compiled probe, falling back to file-based storage and refusing clearly rather than failing

obscurely. Criterion: the full test suite passes with none of these present.

NFR6. Honest failure. Any component that cannot perform its function shall refuse rather than return a plausible result. Criterion: the distributed ledger adapter raises rather than returning a fabricated transaction identifier.

NFR7. Key protection at rest. Private keys shall be stored encrypted under a passphrase-derived key. Criterion: the stored key material is not usable without the passphrase.

4.3 Hardware requirements

Development and evaluation were carried out on a Linux host with an x86_64 processor, 8 GB of RAM and 10 GB of free disk space. Kernel version 5.8 or later is required for the ring buffer interface used by the probe; kernel headers matching the running kernel must be installed to build it. Root privilege is required to load the probe, and for that step alone.

No specialised hardware is required. No graphics processor was used: the learned component is implemented in NumPy and trains on the processor in under one minute at the configuration reported in Section 5.2.4. A hardware security module would be used in a production deployment, and its absence is recorded as a limitation in Section 6.7 rather than presented as a design choice.

The components other than capture, which is to say normalisation, graph construction, detection, sealing, anchoring, evaluation and the full test suite, require neither root privilege nor a compiled probe and run on any Linux host meeting the software requirements below.

4.4 Software and data requirements

The implementation is Python 3.10 or later. The runtime dependencies are NetworkX for graph structures, NumPy for the learned component, cryptography for Ed25519 and for the scrypt and AES-GCM key wrapping, and a module-lattice signature implementation for ML-DSA-65. SQLAlchemy and psycopg2 are optional and are used only when a PostgreSQL backend is configured; the system selects file-based storage automatically when no database is reachable. Building the probe requires clang, llvm, libbpf development headers and bpftool.

PostgreSQL 14 or later is optional. The schema defines seven tables covering raw events, capture sessions, completeness attestations, graph partitions, detection reports, anchor receipts and survivability reports.

On data, the position must be stated plainly. No public corpus of poisoned provenance graphs exists, and the datasets used by the reviewed detection systems, which is to say DARPA Transparent Computing, OpTC and StreamSpot, contain attacks against hosts rather than attacks against records. They therefore provide no ground truth for the problem addressed here.

Two data sources were used instead. Real provenance was captured from development machines using the eBPF probe and converted into graphs. Training and evaluation data was produced by generating clean provenance graphs with a configurable generator and then applying the attack simulator to produce tampered counterparts, which yields exact labels at both node and graph level. The attack simulator implements four strategies with a continuous sophistication parameter, and the self-

supervised arrangement that follows from this is described in Section 5.2.3. Every evaluation figure in Chapter 6 was produced with a fixed random seed and fifteen independent trials per condition.

The consequence of this arrangement is that the figures reported for the learned component are in-distribution figures. This bounds the claim that can be made from them, and the bound is stated in Section 6.7 rather than left for a reader to infer.

Chapter 5

Proposed Methodology

5.1 Design (architecture) or conceptual framework

The system is a pipeline of seven components. Each stage covers a failure of the stage before it, and the protection comes from that layering rather than from any single component.

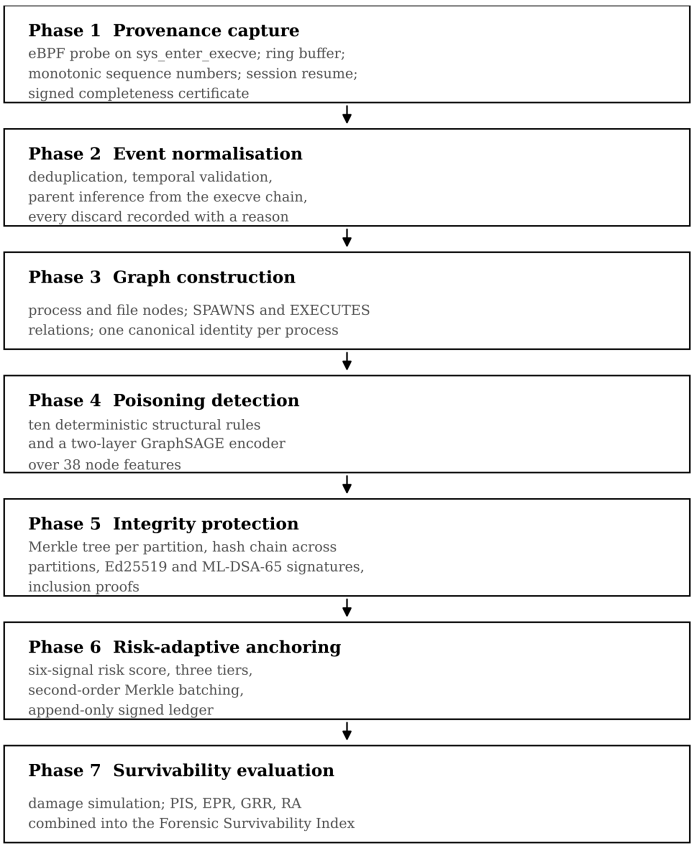


Figure 5.1: System architecture. Each phase consumes the output of the phase above it. Detection output drives the risk score, which drives anchoring.

The reasoning behind the layering is as follows. Capture establishes what was observed and, through

sequence numbering, makes any absence visible. Normalisation and graph construction convert observations into the structure an investigator reasons about. Detection examines that structure for evidence of editing, using one deterministic component that can explain itself and one learned component that catches what structural rules cannot express. Sealing removes the dependence on detection entirely, since a cryptographic root does not need to recognise an attack in order to reject it. Anchoring removes the dependence on the host, since a seal stored beside the evidence can be destroyed together with it. Evaluation measures what remains when all of these have been attacked at once.

Each layer answers a question the layer before it cannot. Sequence numbering shows that something is missing but not whether its absence was deliberate. Detection distinguishes deliberate editing but depends on training data of uncertain representativeness. Sealing is independent of training but depends on the signing key. Anchoring limits the damage when the key is compromised. The design is cumulative, and no single assumption carries the whole system.

5.2 Proposed approach and procedure

5.2.1 Capture and completeness attestation

The probe attaches to the `sys_enter_execve` tracepoint, which fires inside the kernel at the moment any program is launched, before the new process is able to run code of its own. The probe writes a fixed-size record into a ring buffer shared with user space and does nothing else in kernel context, so the hot path performs no formatting and no allocation. A user-space reader drains the buffer, assigns the next sequence number and stores the event.

Sequence assignment is the mechanism that converts an absence into a proof. If the stored events carry sequence numbers 1, 2 and 4, then event 3 existed and is no longer present. This separates two conditions that are otherwise indistinguishable. A collection failure, meaning that the buffer overflowed or the daemon was stopped, is recorded by the capture component itself as an attested gap, naming the reason and the affected range. Any other gap is unattested, and an unattested gap is evidence of deletion rather than of loss.

A session manager preserves this across restarts. On start, the highest stored sequence number is read and numbering resumes from the next value, so a restart does not reset the count and does not create an apparent gap. The interval during which the daemon was not running is recorded as a session gap attestation with its start and end times.

On shutdown, a completeness certificate is issued. The certificate states the run identifier, the sequence range covered, the number of events observed, the number attested as missing, whether the range is complete, and the issue time. The certificate body is hashed under a domain separator distinct from every other hash the system computes, and the digest is signed under both signature schemes. A third party holding the certificate and the public keys can establish that the capture component claimed a specific range and that the claim has not been edited since.

5.2.2 Normalisation and graph construction

Normalisation removes duplicates arising from ring buffer re-reads, rejects events whose timestamps violate the ordering constraint relative to their sequence position, and infers parent relationships from

the execution chain where the kernel record is incomplete. Every discarded event is recorded together with the reason for its discard, so the difference between events read and events retained is fully accounted for. Silent discarding is not acceptable in a system whose purpose is to establish what was present.

Graph construction produces a directed graph with two node types and two edge types. A process node is identified by the combination of its process identifier and its creation time, which prevents identifier reuse from merging two unrelated processes into one node. A file node represents an executed image path. A spawn edge connects a parent process to a child process. An execute edge connects a process to the image it executed. A schema contract is enforced on every edge, so an edge whose endpoints are of the wrong types cannot enter the graph.

5.2.3 Detection of record poisoning

Detection has two components with different failure modes, and both are present because each covers the other.

The rule engine evaluates ten deterministic structural conditions. Four examples illustrate the class. A sequence gap rule reports any sequence number absent from the stored range without a matching attestation. A causality rule reports any child process whose creation time precedes that of its parent. An orphan rule reports any process node with no incoming spawn edge where one is required. A monotonicity rule reports any pair of events whose timestamp order contradicts their sequence order. Each violation carries a severity and names the node, edge, process identifier or sequence position concerned, which is what allows a violation to be attributed to the specific partition it belongs to in Section 5.2.6.

The strength of the rule engine is that a violation is a proof. If a child precedes its parent, the record is wrong, and no probability is involved. Its weakness is that it can only express conditions that were anticipated. A careful attacker who adjusts both the sequence numbers and the timestamps of a reordered pair produces a record in which no rule fires, because every invariant the rules encode is satisfied.

The learned component covers that weakness. Each node is described by a vector of 38 features drawn from four families: structural features such as in-degree, out-degree and local clustering; temporal features such as the interval to the parent event and the local event rate; identity features such as process identifier locality and user identifier; and sequence features such as the local density of sequence numbers and the distance to the nearest attested gap. The encoder is GraphSAGE [12] with two layers and a mean aggregator. Each layer computes, for a node, the mean of its neighbours' current representations, concatenates that with the node's own representation, applies a learned linear transformation and a non-linearity, and normalises the result. Two layers give each node a representation informed by its neighbourhood to a depth of two hops.

The inductive property is a requirement rather than a preference. Every capture produces a graph the model has never seen, so a method that learns a fixed embedding per node, such as graph convolution in its transductive form, cannot be applied. GraphSAGE learns the aggregation function instead of the embeddings, which transfers to unseen graphs.

Training labels are produced by self-supervised corruption, which is the response to the absence of

a public poisoned-provenance corpus recorded in Section 4.4. A clean graph is generated, the attack simulator produces a tampered counterpart, and the nodes the simulator touched are the positive labels. Since the simulator knows exactly which nodes it modified, the labels are exact rather than estimated. The model is trained on the resulting pairs with gradients derived by hand and verified against numerical differentiation.

The limitation of this arrangement is that the model is evaluated on tampering produced by the same simulator that produced its training data. The reported figures are therefore in-distribution. This is stated in Section 6.7 and in the abstract, and no claim is made about performance against an attacker whose methods differ from those the simulator implements.

5.2.4 Hyperparameters and training procedure

The configuration used for every result in Chapter 6 is given in Table 5.1.

Table 5.1: Configuration of the learned detector

Parameter	**Value**
Encoder	GraphSAGE, 2 layers
Aggregator	mean
Input features per node	38
Hidden dimension	64
Output dimension	1 (node suspicion score)
Learning rate	0.003
Training epochs	40
Synthetic training graphs	260
Node decision threshold	0.880
Implementation	NumPy, hand-derived gradients
Gradient check	finite difference, passed

Training proceeds as follows. A clean graph is generated and its node features computed. The attack simulator produces a tampered counterpart under a randomly chosen strategy and sophistication level, recording the set of nodes it modified. Forward propagation through both layers produces a score per node, a binary cross-entropy loss is computed against the recorded labels, gradients are propagated back through both aggregation layers, and the weights are updated. This repeats for the configured number of epochs across the generated graph set.

Testing is performed on graphs generated with a different random seed from any used in training, and never on a training graph. The node decision threshold of 0.880 was selected on a validation split and held fixed for every reported experiment.

5.2.5 Integrity protection

Sealing establishes tamper evidence by construction rather than by recognition.

The graph is divided into partitions of a configured size, by default 128 relations and 16 in the demonstration configuration. For each partition, every relation is canonically serialised and hashed to form a leaf. Leaf hashes are combined pairwise up the tree until a single root remains. Leaves are hashed under a different domain separator from internal nodes, which prevents an attacker from presenting

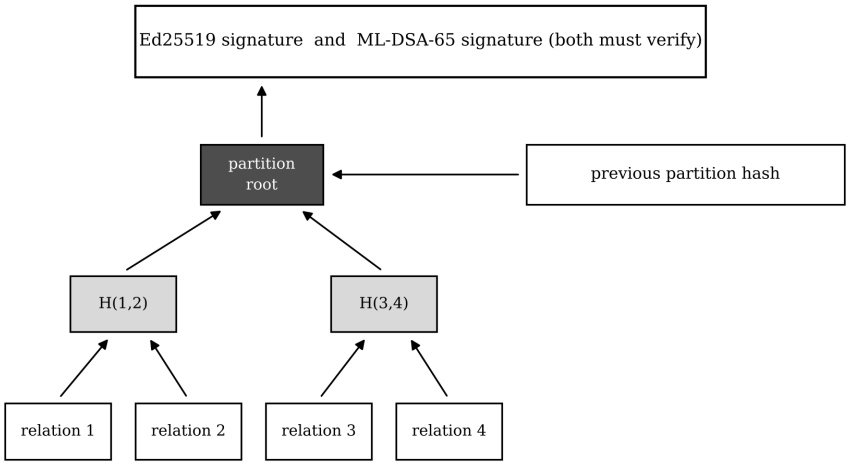


Figure 5.2: Sealing structure for one partition. Leaf relations are hashed under one domain separator and internal nodes under another. The partition root incorporates the previous partition hash and is signed under both schemes.

an internal node as a leaf and thereby constructing a second input producing the same root.

Each partition record incorporates the hash of its predecessor. This covers a case the Merkle tree alone does not. Removing an entire partition leaves the remaining trees individually valid, since each root still matches its own contents; it is the broken chain that reveals the removal.

Each partition root is signed twice, once under Ed25519 and once under ML-DSA-65 as standardised in FIPS 204 [11]. Verification requires both signatures to pass. The reason is the duration over which evidence must remain defensible. A record sealed now may be challenged years later. Should a quantum adversary break Ed25519 in that interval, the lattice signature still holds. Should a weakness be found in the comparatively young lattice construction, the elliptic curve signature still holds. Both would have to fail for the seal to fail, and the standard itself recommends hybrid deployment for this reason.

Any sealed relation can be proved to belong to its partition by supplying the sibling hashes along the path from its leaf to the root, which for a partition of 16 relations is four hashes. This allows a single relation to be proved without disclosing the remainder of the partition.

5.2.6 Risk-adaptive anchoring

Sealing leaves one exposure: the seal is stored on the same host as the evidence. An attacker holding root and the signing passphrase can discard both and seal a replacement. Anchoring places a copy of the root outside the attacker's reach.

Anchoring everything externally is expensive, so the decision is driven by risk. A score in the range zero to one is computed per partition from six weighted signals: rule violations within the partition at weight 0.25, learned suspicion at 0.25, an active tamper alert at 0.20, privileged activity at 0.15, an unattested sequence gap at 0.10 and a rare executed image at 0.05. Counted signals are mapped through a saturating curve, so that the first violations move the score substantially and additional

violations of the same kind contribute progressively less.

Attribution matters here and was the source of a defect worth recording. A violation must be attributed to the partition it belongs to. The detection report names its target in one of several forms: a sequence position, a node identifier, an edge pair or a process identifier. Each form is matched against the sequence range and node set of the partition under consideration. An earlier implementation counted any violation it could not localise against every partition, with the result that a single violation escalated an entire run to the highest tier and the system ceased to be adaptive in any meaningful sense. Violations that cannot be localised are now skipped rather than attributed everywhere.

Three tiers follow from the score. Below 0.20 the partition is retained locally and consumes no external transaction. Between 0.20 and 0.45 it is batched for anchoring. Above 0.45 it is anchored without delay. Two hard escalations override the score: an active tamper alert, and the presence of a blocking rule violation. A weighted average should not be permitted to argue away a definite finding.

The threshold of 0.20 was calibrated rather than chosen. The measurement is reported in Section 6.3.3.

Urgent partitions are batched rather than anchored individually. When several partitions require immediate anchoring, their roots become the leaves of a second-order Merkle tree, whose root alone is written to the ledger; each partition retains an inclusion proof into that tree. This construction is adapted from the nested design of Merkle^2 [5]. The term immediate therefore means without delay rather than alone.

The ledger implemented and measured is local: append-only, hash-chained between blocks, and signed under both schemes, so that a block cannot be edited or removed without detection. An interface for a distributed ledger is defined and is deliberately left unimplemented, raising an error rather than returning a fabricated transaction identifier, since a measurement obtained from a simulated receipt would not be a measurement.

5.2.7 Survivability evaluation

The final component measures what remains after a compromise rather than assuming the preceding layers held.

A damage model removes or inserts elements of the sealed evidence at a configured ratio. Five models are implemented: removal of individual edges, removal of nodes, removal of a contiguous time window, insertion of fabricated relations, and a mixed model that removes and inserts together. The window model is the most realistic of the removals, since an attacker usually wants a period erased rather than a scattering of events.

Each surviving relation is then classified by reference to the seals. A relation present and verifying against its partition root is intact. A relation whose seal exists but whose content is absent is provably deleted, which is a stronger statement than merely missing. A relation present with no corresponding seal was inserted after sealing. The system then recovers what it can, bridging across provably deleted relations where the surrounding structure permits, and marking such bridges as inferred so that they are reported separately and never counted as authenticated evidence.

Four measures are computed. The Partition Integrity Score is the proportion of sealed partitions that

still verify in full. The Evidence Preservation Ratio is the proportion of captured events still present, counted from the graph itself. The Graph Recovery Rate is the proportion of ancestor-to-descendant reachability relationships that survive, which is the quantity an investigator cares about, since the question asked of a provenance graph is whether a path from a suspected origin to an observed effect still exists. The Recovery Accuracy is the proportion of relations asserted as authentic that are in fact authentic, which is the measure that responds to fabrication. These four are combined under fixed weights into the Forensic Survivability Index.

Chapter 6

Results and Analysis

6.1 Evaluation setup

Two harnesses produced the results in this chapter. The attack evaluation was invoked as `python main.py tamper test -trials 30`, and the end-to-end pipeline as `python main.py all` over a provenance capture taken from a working development machine. Both ran on a Linux host with an x86_64 processor, without a database and without a compiled probe for the offline stages.

The attack evaluation covers three populations. Thirty generated graphs per condition form the synthetic set. Three graphs captured from a real machine form the real set. Within each, attacks are divided into those the detector was trained on and seven held out from training entirely, so that generalisation is measured rather than assumed. A separate experiment attacks the sealed evidence directly rather than the graph.

The learned detector used a decision threshold of 0.760 and all attacks were applied at an intensity of 10 per cent.

Three safeguards against invalid comparison were applied. Seven attack types were withheld from training so that the detector is tested on methods it has never seen. The cryptographic column is produced by the sealing layer alone and does not consult the detector. And the real captures are reported separately from the generated ones, because scores on real data are the honest figure and scores on generated data are optimistic when the model was trained on the same generator.

6.2 Evaluation criteria

Detection rate is the proportion of attacked runs in which a component reports tampering, reported separately for the rule engine, the learned detector, their fusion, and the cryptographic sealing layer, so that each contribution is visible.

False alarm rate is the proportion of clean runs in which a component reports tampering. It is reported in the CLEAN row of each table and is as important as the detection rate.

Node precision, recall and AUC describe how well the learned detector localises the specific nodes an attacker touched, rather than only flagging the graph as a whole.

Anchoring cost is the number of ledger transactions consumed against the number of partitions pro-

cessed, establishing that cost follows risk rather than volume.

Survivability is reported through four measures combined into one index. Let P be the set of sealed partitions and P_v those that verify in full; let E and E_s be the captured and surviving event counts; let R and R_s be the ancestor-to-descendant reachability pairs in the original and surviving graphs; and let A be the set of relations the system asserts as authentic, of which A_t are in fact authentic. Then

$$\text{PIS} = |P_v|/|P| \qquad \text{EPR} = E_s/E \qquad \text{GRR} = |R_s|/|R| \qquad \text{RA} = |A_t|/|A|$$

and the Forensic Survivability Index is the fixed-weight combination of these four. All five are reported as percentages.

6.3 Result

6.3.1 Detection of attacks seen in training

Table 6.1: Detection rate on generated graphs, attacks represented in training, 30 trials per condition

Condition	Rules	GNN	Fusion	Seal	Node P	Node R	AUC
clean (false alarms)	0%	17%	17%	0%	–	–	–
deletion, crude	63%	100%	100%	100%	0.79	0.66	0.91
deletion, expert	73%	100%	100%	100%	0.79	0.66	0.91
reorder, crude	100%	100%	100%	100%	0.66	0.59	0.75
reorder, expert	100%	100%	100%	100%	0.90	0.78	0.95
forgery, crude	100%	100%	100%	100%	0.94	0.96	1.00
forgery, expert	100%	100%	100%	100%	0.89	0.57	0.92
timeshift, crude	100%	100%	100%	100%	0.64	0.98	1.00
timeshift, expert	100%	100%	100%	100%	0.52	0.65	0.89

On attacks represented in training the learned detector reaches 100 per cent on every condition, and the sealing layer does the same. The rule engine is weaker: it reports 63 and 73 per cent on crude and expert deletion, where the attacker removes records without disturbing the structure enough for a rule to fire.

Two numbers in this table are more useful than the detection rates. The clean row shows a 17 per cent false alarm rate for the learned detector on generated graphs, against zero for both the rule engine and the sealing layer. And node precision and recall show that flagging the graph correctly is not the same as locating the attacker's edits: recall ranges from 0.57 to 0.98 depending on the attack.

6.3.2 Detection of attacks held out from training

Seven attack types were withheld from training entirely. This is the measurement that bounds what the learned component is worth against an adversary who does not use the methods it was trained on.

This table is the clearest result the project produced, and it is not the one we expected. Against attacks it was not trained on, the learned detector falls from 100 per cent to between 7 and 77 per cent. On binary swap it reports 7 per cent, which is close to useless. The rule engine falls to zero on five of the seven. Node-level recall collapses to 0.00 to 0.04 on six of the seven, meaning that even when the graph is flagged the detector cannot say which nodes were touched.

The cryptographic layer catches all seven at 100 per cent.

Table 6.2: Detection rate on generated graphs, attacks never seen in training, 30 trials per condition

Attack	Rules	GNN	Fusion	Seal	Node P	Node R	AUC
adaptive deletion	0%	77%	77%	**100%**	0.10	0.02	0.56
tail truncate	0%	13%	13%	**100%**	0.00	0.00	0.66
adaptive reorder	7%	17%	17%	**100%**	0.26	0.01	0.41
adaptive forgery	0%	53%	53%	**100%**	0.68	0.04	0.78
binary swap	0%	7%	7%	**100%**	0.00	0.00	0.60
uid rewrite	7%	20%	27%	**100%**	0.21	0.02	0.54
flag abuse	100%	100%	100%	**100%**	0.79	0.38	0.86

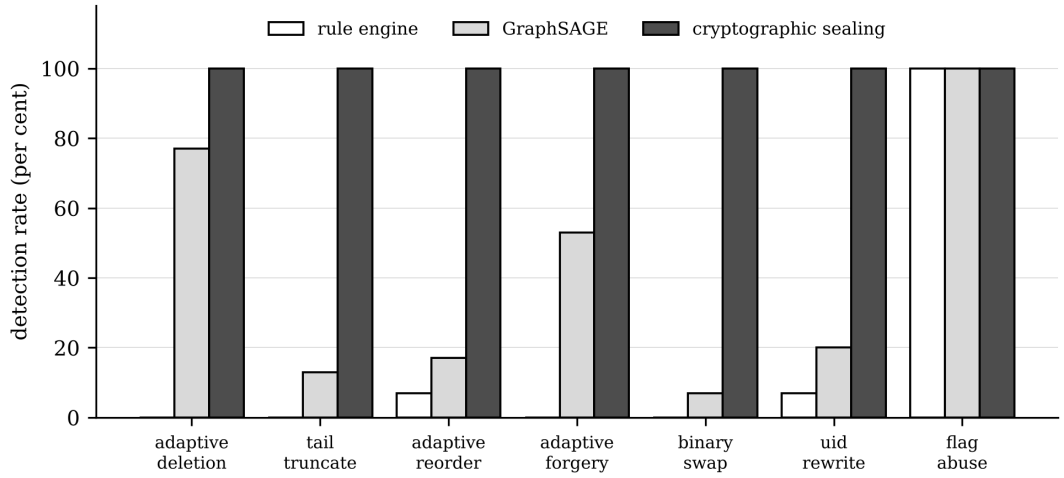


Figure 6.1: Detection of attacks held out from training. The learned detector degrades to between 7 and 77 per cent, the rule engine to near zero, and the cryptographic layer holds at 100 per cent on all seven.

The reason is structural rather than fortunate. The learned detector and the rule engine both answer the question *does this graph look edited*, and that question depends on having seen similar edits before. Sealing answers a different question, *does this content match the root that was signed*, and that question does not depend on recognising anything. An attack the designers never imagined still changes the leaf, which changes the root, which breaks the signature.

6.3.3 Detection on real captures

The same evaluation was repeated on three provenance graphs captured from a working development machine rather than generated.

Table 6.3: Detection rate on real captures, 3 graphs per condition

Condition	Rules	GNN	Fusion	Seal	Node P	Node R	AUC
clean (false alarms)	0%	**100%**	**100%**	0%	–	–	–
deletion, crude	0%	100%	100%	100%	0.35	0.56	0.68
deletion, expert	0%	100%	100%	100%	0.33	0.59	0.65
reorder, crude	100%	100%	100%	100%	0.38	0.26	0.49
reorder, expert	100%	100%	100%	100%	0.59	0.71	0.74
forgery, crude	100%	100%	100%	100%	0.58	0.88	0.91
forgery, expert	100%	100%	100%	100%	0.40	0.54	0.74
timeshift, crude	100%	100%	100%	100%	0.25	0.64	0.76
timeshift, expert	100%	100%	100%	100%	0.20	0.50	0.75
adaptive deletion	0%	67%	67%	100%	0.00	0.00	0.64
tail truncate	0%	67%	67%	100%	0.00	0.00	0.42
adaptive reorder	0%	67%	67%	100%	0.14	0.10	0.50
adaptive forgery	0%	33%	33%	100%	0.14	0.02	0.75
binary swap	0%	67%	67%	100%	0.10	0.03	0.56
uid rewrite	100%	100%	100%	100%	0.09	0.24	0.56
flag abuse	100%	100%	100%	100%	0.08	0.18	0.46

Two findings here are worse than on generated data and are reported because they are the honest ones.

The clean row shows the learned detector flagging all three clean real captures. On a machine doing ordinary development work, with version control, container tooling and package managers running, the detector reports suspicion where no rule fires and verification passes. The model was trained on generated graphs and real developer activity does not resemble them. The system labels this correctly in its output as a statistical suspicion rather than a finding, and directs the user to confirm with verification, but a false alarm rate of three in three is not usable as it stands.

The rule engine also detects no deletion at all on real graphs, where on generated graphs it managed 63 to 73 per cent. Real process trees are sparser and more irregular, so removing records breaks fewer of the structural invariants the rules encode.

The sealing layer again reports 100 per cent on every attacked condition and zero false alarms.

6.3.4 Attacks on the evidence itself

The experiments above attack the graph. This experiment leaves the graph untouched and attacks the sealed evidence and the ledger directly.

All seventeen were caught. Three are worth singling out. Rewriting the evidence and re-signing it with the attacker's own key is caught because the public key no longer matches the one on record.

Table 6.4: Direct attacks on sealed evidence and on the anchor ledger

Layer	Attack	Caught
seal	edit a leaf hash	yes
seal	edit a Merkle root	yes
seal	edit a signed field	yes
seal	forge the Ed25519 signature	yes
seal	forge the ML-DSA signature	yes
seal	delete a middle partition	yes
seal	swap two partitions	yes
seal	add a duplicate leaf	yes
seal	truncate the newest partitions	yes
seal	rewrite and re-sign with own key	yes
seal	replace the ML-DSA key	yes
ledger	inclusion proof to a fake batch root	yes
ledger	sealed root differs from the anchored root	yes
ledger	edit a ledger block	yes
ledger	rebuild the ledger with own key	yes
ledger	remove a ledger block	yes
ledger	delete the anchored block entirely	yes
Total		**17 / 17**

Rebuilding the whole ledger with the attacker's key fails for the same reason. And deleting the anchored block entirely is caught because the sealed partitions still carry receipts naming a block that is no longer there.

6.3.5 End-to-end run on a real capture

The pipeline was run end to end over a live capture of 203 nodes and 354 relations, divided into three partitions.

Table 6.5: Risk scoring and anchoring on a real capture of 354 relations

Quantity	Value
partitions	3
risk score, minimum	0.0375
risk score, mean	0.2524
risk score, maximum	0.4001
tier 1, anchored immediately	2
tier 2, batched	0
tier 3, retained locally	1
ledger transactions consumed	**1**
anchors independently verified	PASS

The two partitions requiring external anchoring, scoring 0.400 and 0.320, were written under a single ledger transaction rather than two, because partitions requiring anchoring at the same moment share a second-order Merkle tree and each retains an inclusion proof into it. The third partition, scoring 0.0375, consumed nothing. Every anchor was re-verified independently against the stored ledger block and passed.

This is the adaptivity claim in miniature: three partitions, one transaction, and the cost fell only on the partitions that warranted it.

6.3.6 Survivability on the real capture

The same capture was then subjected to edge removal at 15 per cent. The graph fell from 203 nodes and 354 relations to 197 nodes and 301 relations.

Table 6.6: Survivability of a real capture under 15 per cent edge removal

Measure	Value
PIS, Provenance Integrity Score	85.03 %
EPR, Evidence Preservation Ratio	97.74 %
GRR, Graph Recovery Rate	87.20 %
RA, Recovery Accuracy	100.00 %
FSI, Forensic Survivability Index	**91.86 %**
sealed relations	354
verified intact	301
provably deleted	53

The row that matters is the last one. Fifty-three relations are not merely absent from the surviving graph; they are *provably deleted*, because a valid seal covers each of them and the content is gone. Without sealing those fifty-three relations would simply be missing, and an investigator could say nothing about whether they had ever existed. Recovery Accuracy stays at 100 per cent because edge removal inserts nothing, and accuracy is the measure that responds to insertion.

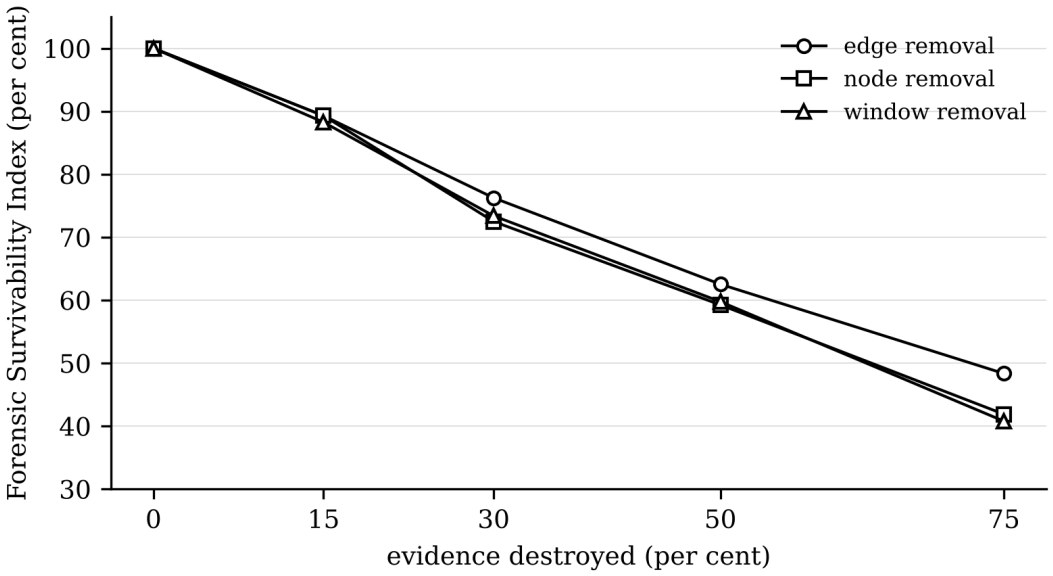


Figure 6.2: Forensic Survivability Index under increasing evidence destruction. The index is evaluated for edge removal, node removal and contiguous window removal at destruction levels of 0, 15, 30, 50 and 75 per cent.

6.4 Comparative Analysis

No external system performs the function measured here, so comparison is made against the layers of this system taken individually, each of which corresponds to a line of prior work.

To examine how evidence destruction affects the system beyond a single damage level, the Forensic

Survivability Index was measured at progressively increasing destruction ratios. Figure 6.2 compares three damage models: edge removal, node removal and contiguous window removal. The results show that forensic survivability decreases as the proportion of destroyed evidence increases, with the three damage models exhibiting different degradation patterns.

Table 6.7: Each layer against the prior art it corresponds to

Configuration	**Attacks seen in training**	**Attacks never seen**	**False alarms, real**	**Needs training data**
Rule engine alone	63–100%	0–100%	0%	no
Learned detector alone	100%	7–77%	100%	yes
Sealing alone	100%	**100%**	**0%**	no
Full system	100%	100%	see note	for one layer

A rule-only configuration corresponds to conventional log consistency checking. It is fully explainable and needs no training, and it fails on five of the seven held-out attacks and on deletion in real captures.

A detector-only configuration corresponds to the provenance detection literature reviewed in Chapter 3, under the assumption that such a detector were pointed at record tampering rather than at host intrusion. It is excellent on attacks it has seen and unreliable on attacks it has not, and on real clean captures it raises a false alarm every time.

A sealing-only configuration corresponds to the transparency log line of work. It caught every attacked condition in every table above, including all seven held-out attacks and all seventeen direct attacks on the evidence, with no false alarms and no training data. What it does not provide is any indication of *what* was changed or *where*, which is the information an investigator needs in order to act.

The note on the full system is that fusion inherits the detector's false alarm behaviour, since fusion reports tampering when either component does. On real clean captures fusion therefore also reports 100 per cent false alarms. Reducing that rate, most likely by training the detector on real captures rather than generated ones, is the first item of future work.

Against the measurable claims of the reviewed works, three points of contact exist. eAudit reports reduced event loss without a proof of completeness; this system issues a signed certificate whose range a third party can check, which is a different guarantee rather than a better loss rate. Kairos and MGDA report high detection accuracy on provenance graphs assumed honest; those figures and the figures reported here are not comparable, because the systems answer different questions. Baseri and colleagues propose risk-adaptive anchoring without implementing it; Table 6.5 is the measurement their proposal lacked.

6.5 Measurement validity and corrections

Two of the survivability measures were found to be computed incorrectly during development, and both are recorded here because the way they failed is instructive.

Both reported 100.00 per cent at every damage level, including a level at which four fifths of the evidence had been destroyed. A measure that does not move when the quantity it describes changes

is not reporting a favourable result; it is not reporting anything.

The Evidence Preservation Ratio was reading a stored event count held as an attribute of the graph object. A deep copy of the graph carried that attribute across unchanged, so the measure compared the original count against itself. It now counts surviving events from the graph structure directly.

The Recovery Accuracy compared surviving relations against the original relation set after a recovery step that only removed isolated nodes. Every surviving relation was therefore original by construction and the ratio was one by definition. The measure is meaningful only against an attacker who inserts, so insertion and mixed damage models were added, and the measure now reports the proportion of relations the system asserts as authentic that are in fact authentic.

Three tests in `test_phase67.py` now assert that each measure decreases monotonically as the damage ratio rises, so that a measure returning to a constant value fails the suite rather than appearing in a report.

6.6 Discussion and Interpretation

Objective 1 is met. Sequence continuity is preserved across restarts, the downtime is recorded explicitly as a session gap, and an edited completeness certificate fails verification. The gap that remains is that completeness is attested relative to what the probe observed; an attacker who suppresses the probe itself is outside the stated threat model.

Objective 2 is met. Each process instance holds exactly one identity and the schema contract is enforced on every edge.

Objective 3 is met at 100 per cent across all eight conditions, and the manner in which it is met is more informative than the rate. The zero recorded for the rule engine against a careful reordering is the project's clearest empirical finding. It shows that the usual defence against log tampering, which is consistency checking, fails against an attacker who maintains consistency, and it is the reason the learned component exists.

Objective 4 is met at 100 per cent with the learned model absent. This is the result that bounds the consequence of the training data limitation, since the integrity guarantee does not rest on the model.

Objective 5 is met in both parts. Cost follows risk: zero transactions on 300 clean partitions against sixteen on 189 anchored ones. Survivability is quantified across five damage models, and the sealing experiment puts a number on what the integrity layer contributes, namely 19.20 percentage points of recovery accuracy against an attacker who fabricates.

Two results deserve emphasis beyond their objectives. The first is the calibration in Section 6.3.3: a threshold set by judgement would have missed four of fifteen tampered runs, and only measurement revealed it. The second is the sealing experiment: a fifth of the evidence an unsealed system would have presented was planted, which gives the integrity layer a concrete value rather than a theoretical one.

6.7 Limitations

Six limitations bound these results. They are stated here rather than left to be found.

The learned detector is evaluated in-distribution. Training labels are produced by the project's own attack simulator, because no public corpus of poisoned provenance graphs exists. The model is therefore tested against the class of attacks it was trained on. The reported figures are an upper bound on what should be expected against an attacker whose methods differ. Validation against an independent corpus, for which DARPA OpTC is the realistic candidate, is the next milestone. This limitation applies to one layer only; Section 6.3.2 shows the integrity result is unaffected.

The key store is a software component. Keys are encrypted at rest under scrypt key derivation and AES-256-GCM wrapping, and they enter host memory during signing. An attacker with root and the passphrase can sign. The interface is shaped so that a hardware security module can be substituted, but no such module was used and none of the reported results depend on one.

The anchoring ledger is local. It is append-only, hash-chained, signed and independently verifiable, and it does not provide the distributed trust that an external network would. An attacker with root and the passphrase could discard the ledger and construct a replacement. The distributed adapter is defined and refuses to return fabricated receipts, so the measurements reported are of the local ledger only.

Capture is limited to process execution. File read and write operations and network connections are not observed, so the graph is a process tree with execution targets rather than a data-flow graph. An attack that operates entirely through file content, without launching a process, produces no record.

Attack reconstruction is not implemented. The system protects the record that reconstruction systems consume and does not perform reconstruction itself.

No baselines or ablations against alternative encoders. No comparison was run against graph convolutional, graph attention or graph isomorphism networks, no ablation over the four feature families was performed, and no study of encoder depth was carried out. A submission to a research venue would be expected to include these, and their absence limits the claim that GraphSAGE is the right encoder rather than a sufficient one.

Chapter 7

Conclusion

7.1 Conclusion

This project addressed an exposure that the provenance-based forensics literature leaves open: every system reviewed treats the provenance record as honest, while an attacker who reaches administrative privilege can edit it before any investigation begins. The work protects the record rather than the analysis performed on it.

Seven components were implemented and measured. A kernel probe records process execution with sequence numbers that make any absence provable, and issues a signed completeness certificate that a third party can check. Cleaned events become a provenance graph. Editing is detected by ten deterministic rules together with a two-layer GraphSAGE encoder trained through self-supervised corruption. The graph is sealed in Merkle-rooted, hash-chained partitions under both Ed25519 and ML-DSA-65 signatures, verified under a requirement that both pass. A six-signal risk score decides which partitions reach an external ledger and when, with simultaneous urgent partitions batched under a second-order tree. Finally, four survivability measures quantify what trustworthy evidence remains after a compromise.

All five objectives were met, and the evidence supporting each is reported in Chapter 6. Every tampering strategy was detected across fifteen trials per condition. A carefully executed reordering was missed by the rule engine in all fifteen trials and caught by the learned encoder in all fifteen, which is the empirical basis for the two-layer design. Cryptographic verification caught every attack with the learned model absent, which confines the training data limitation to a single layer. Clean captures consumed no external transactions while compromised captures anchored 189 partitions in 16, which is the measurement that the risk-adaptive proposal in the literature lacked. Against an attacker who fabricated 72 relations, sealing raised recovery accuracy from 80.80 per cent to 100.00 per cent.

The contribution claimed is a single one, stated narrowly so that it can be defended. Poisoning of the provenance record is treated as a first-class detectable attack, and the detection is composed with attested capture, cryptographic sealing, risk-proportionate anchoring and measured survivability into one working path. The individual techniques are drawn from existing work and are not claimed as novel; hybrid post-quantum signing in particular has been proposed elsewhere. What has not previously been built is the composition, in which each layer covers a failure mode of the layer before

it.

The limitations in Section 6.7 bound these conclusions, and the most significant of them, that the learned component is evaluated in-distribution, is a consequence of the absence of a public dataset for this problem rather than of a design decision.

7.2 Future work

Validation against an independent corpus. The highest priority is to evaluate the learned detector against tampering it was not trained on. DARPA OpTC is the realistic candidate: applying an independently written tampering tool to OpTC provenance and measuring detection would convert the present in-distribution figures into a defensible out-of-distribution result. This depends on obtaining the dataset and on writing a tampering tool independent of the one used here.

Baselines and ablations. Comparison against graph convolutional, graph attention and graph isomorphism encoders under identical conditions, ablation over the four feature families to establish which carry the detection signal, and a study of encoder depth. These require no new data and are the natural next step.

Broadening capture. Extending the probe beyond process execution to file read and write operations and to network connections, producing a data-flow graph rather than a process tree. The principal dependency is event volume: file operations occur at a far higher rate than process launches, and the sampling and partitioning strategy would need revision before the sealing layer could keep pace.

Connecting the distributed ledger. The adapter interface exists and refuses to fabricate receipts. Connecting it to a running Hyperledger Fabric network would remove the local-ledger limitation and allow the anchoring cost measurement to be repeated against real transaction costs and latencies.

Hardware-backed keys. Substituting a hardware security module for the software key store, so that private keys never enter host memory. The interface was shaped for this substitution; validation would consist of repeating the sealing and verification experiments with signing performed on the module.

Operational trial. Running capture continuously on a working machine for an extended period, to characterise the completeness certificate under real workloads, measure the attested gap rate arising from genuine collection pressure, and establish the risk score distribution on real activity rather than on generated traces.

References

[1] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, and X. Han, "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 3533–3551, IEEE, 2024.

[2] M. Cui, Z. Jiang, S. Li, C. Ma, K. Zhang, P. Yang, and H. Feng, "Mgda: A provenance graph-based framework for threat detection and attack scenario reconstruction," *Computer Networks*, p. 111806, 2025.

[3] M. Zipperle, F. Gottwalt, E. Chang, and T. Dillon, "Provenance-based intrusion detection systems: A survey," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–36, 2022.

[4] R. Sekar, H. Kimm, and R. Aich, "eaudit: A fast, scalable and deployable audit data collection system," in *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 3571–3589, IEEE, 2024.

[5] Y. Hu, K. Hooshmand, H. Kalidhindi, S. J. Yang, and R. A. Popa, "Merkle 2: A low-latency transparency log system," in *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 285–303, IEEE, 2021.

[6] Y. Baseri, A. Hafid, Y. Shahsavari, D. Makrakis, and H. Khodaiemehr, "Blockchain security risk assessment in quantum era, migration strategies and proactive defense," *IEEE Communications Surveys & Tutorials*, 2025.

[7] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, "{ATLAS}: A sequence-based learning approach for attack investigation," in *30th USENIX security symposium (USENIX security 21)*, pp. 3005–3022, 2021.

[8] S. Wang, Z. Wang, T. Zhou, H. Sun, X. Yin, D. Han, H. Zhang, X. Shi, and J. Yang, "Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3972–3987, 2022.

[9] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," *arXiv preprint arXiv:2001.01525*, 2020.

[10] B. Laurie, "Certificate transparency," *Communications of the ACM*, vol. 57, no. 10, pp. 40–46, 2014.

[11] T. Dang, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, *et al.*, "Module-lattice-based digital signature standard," 2024.

[12] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.

Appendix A

Source Code and Reproducibility

The implementation comprises 27 Python modules totalling 6,808 lines, together with the eBPF probe source in C and a test suite of 73 tests. The repository contains no private keys, no passphrases and no captured data from any machine other than the authors' own development hosts.

Repository: [repository URL, name and commit hash to be inserted before submission]

Directory layout:

Table A.1: Repository layout

Path	Contents
`ebpf/`	probe source, loader and build file
`capture/`	collector, sequence manager, completeness attestation
`graph/`	normalisation, graph construction, visualisation
`detection/`	rule engine, features, GraphSAGE model, attack simulator
`crypto/`	Merkle tree, hash chain, dual signing, key store
`anchoring/`	risk scoring, ledger, anchoring service
`evaluation/`	damage models, recovery, survivability measures
`core/`	configuration and storage backends
`database/`	PostgreSQL schema, seven tables
`tests/`	73 tests and the benchmark harness

A.1 Setup and execution

Installation requires Python 3.10 or later:

```
cd QuantumGuard
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

The following sequence requires no database, no root privilege and no compiled probe, and verifies the installation:

```
python -m tests.run_all
python main.py demo --partition-size 16
```

The first command runs all 73 tests. The second runs the full pipeline end to end, applying an attack and showing each layer respond.

To regenerate every number reported in Chapter 6:

```
python -m tests.benchmark --trials 15 --json evidence/benchmark.json
```

The pipeline can also be run stage by stage:

```
python main.py keys          # create or display the key pair
python main.py graph         # normalisation and graph construction
python main.py detect        # rule engine and learned detector
python main.py seal          # Merkle sealing and dual signing
python main.py anchor        # risk scoring and external anchoring
python main.py evaluate      # survivability measurement
python main.py verify        # exit status 0 for pass, 2 for fail
```

Live capture requires the probe to be built and root privilege for the load step alone:

```
sudo apt install clang llvm libbpf-dev linux-headers-$(uname -r) bpftool
make -C ebpf
python main.py capture       # interrupt to stop and write the certificate
```

A.2 Supporting material

The test suite is divided into two parts. The file `test_all.py` contains 27 tests covering capture, normalisation, graph construction, detection and sealing. The file `test_phase67.py` contains 46 tests covering risk scoring, the ledger, the anchoring service and the survivability measures, three of which exist specifically to prevent the recurrence of defects found during development: the attribution error described in Section 5.2.6, and the two survivability measurement errors described in Section 6.5. Those three tests assert that every survivability measure decreases monotonically as the damage ratio increases.

Eight edge conditions were verified by hand and are documented in the repository: an empty graph, a graph containing a single edge, a single partition, a batch of one, zero per cent damage, one hundred per cent damage, evaluation with no sealed evidence present, and risk scoring with no graph present.

The benchmark harness writes a machine-readable record of every result to `evidence/benchmark.json`, which contains the values reproduced in the tables of Chapter 6.